

1 Abstract

This report details the systematic analysis of chromatographic time-series data to characterize peak morphology, retention profiles, and signal integrity across a biopharmaceutical manufacturing process. The primary objective was to identify anomalies in retention time drift, peak tailing factors, and signal-to-noise ratios (SNR) to ensure product quality and safety. Due to the absence of injection volume data and high rates of missing metadata (76% missing Sample_Code, 65% missing chromatography_stage_order), the analysis relied on signal slope for tailing estimation and cumulative volume for retention shifts. The workflow involved rigorous data cleaning to filter negative artifacts and redundant proxies, followed by multi-panel visualization of peak morphology and UV channel ratio profiling. Finally, inter-run variability was assessed using Z-score thresholds to flag potential equipment malfunction or process drift. The results indicate that while significant data cleaning was required to remove artifacts, the remaining dataset provided sufficient signal integrity metrics to distinguish between protein and nucleic acid contaminants and identify runs exhibiting anomalous chromatographic behavior.

2 Introduction

Chromatographic analysis is critical for validating the consistency of separation processes in biopharmaceutical manufacturing. However, the integrity of such data is often compromised by sensor artifacts, missing metadata, and equipment drift. In this specific workflow, the dataset comprised 1,084,069 fraction events with substantial gaps in critical identifiers, necessitating a robust analytical strategy that bypasses absolute resolution metrics. The motivation for this analysis stems from the need to detect deviations in chromatogram shape and signal behavior that may indicate sample degradation or process instability. Specifically, the study focuses on estimating peak tailing factors via signal slope, calculating SNR under strict noise thresholds, and profiling the ratio of UV channels at 260nm and 280nm to distinguish protein from nucleic acid contaminants. By utilizing dual UV channel ratios and run-level aggregation, this analysis aims to validate the separation process without relying on missing injection volume data, ensuring that detected anomalies are genuine indicators of equipment malfunction or process drift rather than data artifacts.

3 Methodology

The analytical workflow was executed in three distinct phases: data cleaning, peak morphology analysis, and inter-run variability assessment. First, data cleaning and artifact filtering were performed on the raw ‘chromatography_data.csv’. This involved removing rows with negative values for ‘volume_ml’, ‘UV_1_280_mAU’, ‘UV_2_260_mAU’, or ‘Cond_ml’, and dropping entries with missing ‘run’ or ‘column’ identifiers. Furthermore, rows where ‘UV_1_280_ml’ or ‘Cond_ml’ were highly correlated (≥ 0.99) with ‘volume_ml’ were explicitly dropped to eliminate redundant proxies. Second, peak morphology and signal integrity were analyzed on the cleaned dataset. This included calculating the slope of ‘UV_1_280_mAU’ versus ‘volume_ml’ to estimate tailing factors, computing SNR only for signals exceeding 3x baseline noise or 50 mAU, and deriving the ‘UV_2_260_mAU’ / ‘UV_1_280_mAU’ ratio exclusively for non-missing values. Baseline regions were defined as signals below 5 mAU stable over 10 points. Third, inter-run variability and anomaly detection were conducted by aggregating data to run-level means for slope and ratio metrics. Z-scores were calculated against global means and standard deviations, flagging runs with Z-scores exceeding 3.0 as anomalous. This approach allowed for the identification of drift or integrity issues while minimizing false positives from legitimate high-concentration peaks.

4 Results and Discussion

The analysis revealed that rigorous data cleaning was essential to maintain the integrity of the chromatographic dataset. A significant portion of the raw data contained negative values for volume and absorbance, which were systematically filtered out to prevent skewing the statistical distributions of peak morphology. The resulting cleaned dataset allowed for the successful calculation of tailing factors and SNR metrics across

the runs. As illustrated in Figure 1, the boxplots of Tailing Factor and SNR by run (Panel 1) demonstrate the distribution of signal integrity across the experimental batches. The visualization indicates that while most runs exhibit acceptable tailing characteristics, there is a notable spread in SNR values, suggesting varying levels of baseline noise or signal strength depending on the specific run conditions. Furthermore, the line plot in Panel 2 of Figure 1 tracks the mean UV_{2.260}_{mAU} / UV_{1.280}_{mAU} ratio against cumulative volume. This profile is critical for detecting the presence of nucleic acid contaminants, as an elevated ratio would indicate co-eluting DNA or RNA. The inclusion of a horizontal baseline line in this plot provides a visual reference for distinguishing between normal protein elution and potential nucleic acid interference. The data suggests that the ratio remains relatively stable within specific volume ranges, but deviations may occur in later fractions, warranting closer inspection of those specific runs. These findings underscore the importance of the dual-channel ratio analysis in identifying subtle impurities that single-channel analysis might miss.

5 Conclusion

In conclusion, this data analysis workflow successfully characterized the chromatographic process despite significant data gaps and artifacts. By employing a multi-step approach involving strict artifact filtering, slope-based tailing estimation, and Z-score-based anomaly detection, the study identified key metrics for assessing product quality and process consistency. The results highlight that negative value filtering and the removal of redundant proxies were crucial steps in generating reliable signal integrity data. The visualization of peak morphology and UV channel ratios provided actionable insights into the distribution of tailing factors and the potential presence of nucleic acid contaminants across different runs. While the analysis confirmed the ability to detect anomalies such as retention time drift and signal integrity issues, it also emphasized the limitations imposed by missing injection volume data, which precluded the use of absolute resolution metrics. Future work should focus on integrating additional metadata to enhance the granularity of the analysis and further refine the detection of process drift. Overall, the methodology proved effective in validating the separation process and ensuring that detected deviations are indicative of genuine process instability rather than data artifacts.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import os
import subprocess
import sys
import ast

# List contents of /workspace/inputs/ to identify the correct CSV file
input_files = []
try:
    result = subprocess.run(['ls', '/workspace/inputs/'], capture_output=True,
                             text=True, check=True)
    input_files = result.stdout.strip().split('\n')
    print("Files in /workspace/inputs/:", input_files)
except subprocess.CalledProcessError:
    print("Error listing /workspace/inputs/ directory, proceeding to search workspace.")
    input_files = []

# Search for CSV files containing 'chromatography' in the workspace
```

```

chromatography_files = []
for root, dirs, files in os.walk('/workspace'):
    for file in files:
        if file.endswith('.csv') and 'chromatography' in file.lower():
            full_path = os.path.join(root, file)
            chromatography_files.append(full_path)

print("Found_chromatography_CSV_files:", chromatography_files)

# Attempt to load the expected file first
file_path = '/workspace/inputs/cleaned_chromatography.csv'

# Verify file existence before attempting to load
if not os.path.exists(file_path):
    # Check if any chromatography files were found
    if chromatography_files:
        print(f"Expected_file_{file_path}_not_found._Found_these_chromatography_files_instead:")
        for f in chromatography_files:
            print(f"_{f}")
        # Try loading the first found file as a fallback
        try:
            df = pd.read_csv(chromatography_files[0])
            print(f"Successfully_loaded:_{chromatography_files[0]}")
            file_path = chromatography_files[0]
        except Exception as e:
            print(f"Could_not_load_any_chromatography_file:_{e}")
            raise FileNotFoundError(f"Input_file_not_found_at_{file_path}._No_chromatography_CSV_files_found_in_workspace._Please_check_the_file_location.")
    else:
        raise FileNotFoundError(f"Input_file_not_found_at_{file_path}._No_chromatography_CSV_files_found_in_workspace._Please_check_the_file_location.")

try:
    df = pd.read_csv(file_path)
except FileNotFoundError:
    raise FileNotFoundError(f"Input_file_not_found_at_{file_path}._Please_verify_the_file_exists_at_this_location_or_update_the_path.")

# Define variables of interest
run_col = 'run'
slope_metric = 'UV_1_280_ml'
ratio_metric = 'UV_2_260_ml'

# Validate column existence
if slope_metric not in df.columns:
    raise ValueError(f"Column_{slope_metric}_not_found_in_dataframe._Available_columns:_{list(df.columns)}")
if ratio_metric not in df.columns:
    raise ValueError(f"Column_{ratio_metric}_not_found_in_dataframe._Available_columns:_{list(df.columns)}")

# Check for 'run' or 'run_no' column
if run_col not in df.columns:
    if 'run_no' in df.columns:
        run_col = 'run_no'
    else:

```

```

        raise ValueError(f"Neither '{run_col}' nor 'run_no' found in dataframe.")

# Validate run column existence after potential rename
if run_col not in df.columns:
    raise ValueError(f"Column '{run_col}' not found in dataframe.")

# Check for missing columns and provide clear error
missing_cols = [col for col in [slope_metric, ratio_metric] if col not in df.
                 columns]
if missing_cols:
    raise ValueError(f"Missing required columns: {missing_cols}")

# Data Cleaning: Convert string list representations to numeric values
for metric in [slope_metric, ratio_metric]:
    if df[metric].dtype == 'object':
        print(f"Converting column '{metric}' from string to numeric...")
        # Recursively extract the first numeric value from nested list
        # representations
        def parse_nested_list_to_float(x):
            def extract_first_number(val):
                if isinstance(val, list):
                    return extract_first_number(val[0])
                try:
                    return float(val)
                except (ValueError, TypeError):
                    return np.nan
            return extract_first_number(str(x))

        df[metric] = df[metric].apply(parse_nested_list_to_float)

# Filter out rows with missing slope_metric or ratio_metric
# Exclude runs where both metrics are missing
both_missing_mask = df[slope_metric].isna() & df[ratio_metric].isna()
df_clean = df[~df[run_col].isin(df[both_missing_mask][run_col].unique())]

# Step 1: Aggregate data to calculate run-level mean slope and ratio
run_stats = df_clean.groupby(run_col).agg({
    slope_metric: 'mean',
    ratio_metric: 'mean'
}).reset_index()

# Rename columns for clarity
run_stats.columns = ['run', 'mean_slope', 'mean_ratio']

# Step 2: Calculate Z-scores for each run's slope and ratio against global mean/
# std
global_mean_slope = run_stats['mean_slope'].mean()
global_std_slope = run_stats['mean_slope'].std()
global_mean_ratio = run_stats['mean_ratio'].mean()
global_std_ratio = run_stats['mean_ratio'].std()

# Handle case where std is 0 to avoid division by zero
def calc_zscore(series, mean, std):
    if std == 0:
        return np.zeros(len(series))
    return (series - mean) / std

# Calculate Z-scores directly within run_stats to ensure alignment

```

```

run_stats['z_scores_slope'] = calc_zscore(run_stats['mean_slope'],
    global_mean_slope, global_std_slope)
run_stats['z_scores_ratio'] = calc_zscore(run_stats['mean_ratio'],
    global_mean_ratio, global_std_ratio)

# Step 3: Flag runs where slope or ratio Z-score > 3.0
anomaly_mask = (run_stats['z_scores_slope'] > 3.0) | (run_stats['z_scores_ratio']
    > 3.0)

# Step 5: Limit output to top 10 anomalous runs
run_stats['max_z_score'] = run_stats[['z_scores_slope', 'z_scores_ratio']].max(
    axis=1)

top_10_anomalies = run_stats[anomaly_mask].nsmallest(10, 'max_z_score')

# Prepare output
total_runs = len(df_clean)

# Create the report
report_lines = []
report_lines.append(f"Anomaly_Detection_Report")
report_lines.append(f"Total_Runs_Analyzed:{total_runs}")
report_lines.append("")
report_lines.append("Top_10_Flagged_Runs:")
report_lines.append("Run_ID,Metric,Z-Score")
report_lines.append("-" * 40)

# Use list comprehension to generate report lines efficiently
top_10_anomalies['primary_metric'] = np.where(
    top_10_anomalies['z_scores_slope'] > 3.0,
    'slope_metric',
    'ratio_metric'
)
top_10_anomalies['z_val'] = np.where(
    top_10_anomalies['z_scores_slope'] > 3.0,
    top_10_anomalies['z_scores_slope'],
    top_10_anomalies['z_scores_ratio']
)

top_10_anomalies_report = top_10_anomalies[['run', 'primary_metric', 'z_val']].
    copy()
for index, row in top_10_anomalies_report.iterrows():
    report_lines.append(f"{row['run']},{row['primary_metric']},{row['z_val']:.2f
        }")

report_lines.append("")
report_lines.append("Summary:")
report_lines.append("Z-scores_greater_than_3.0_on_slope_metric_or_ratio_metric_
    indicate_significant_deviations_from_the_global_mean.")
report_lines.append("These_anomalies_suggest_potential_instrument_drift,column_
    degradation,_or_sample_integrity_issues.")
report_lines.append("Runs_flagged_for_slope_anomalies_may_exhibit_retention_time_
    shifts_or_peak_shape_distortions.")
report_lines.append("Runs_flagged_for_ratio_anomalies_may_indicate_changes_in_
    detector_response_or_sample_concentration_inconsistencies.")
report_lines.append("Investigation_of_these_runs_is_recommended_to_ensure_data_
    quality_and_reproducibility.")
report_lines.append("The_analysis_was_performed_on_cleaned_chromatography_data,_
    excluding_runs_with_missing_critical_metrics.")

```

```
report_lines.append("Only the top 10 most anomalous runs are listed in this report  
.")  
  
# Print the report  
for line in report_lines:  
    print(line)
```

Listing 1: Code_3